



⇒ 114-2 NCKU Program Design-II HW4

繳交期限

2026年 05月 26日(Tue) 23:59:59

題目說明

真新鎮的少年小智再度啟程！這一次，他來到了充滿強敵的全新賽區。
面對眼前強大的野生頭目(Boss)神奇寶貝，小智必須善用手邊的陣容來進行戰鬥。
在這場生存懸死戰中，每一隻神奇寶貝面對攻擊與反擊時，都有截然不同的行為模式。
下面是小智的神奇寶貝介紹：

神奇寶貝	屬性	弱點	生命值	攻擊力
皮卡丘 (Pikachu)	電 (Electric)	地面 (Ground)	300	80
妙蛙種子 (Bulbasaur)	草 (Grass)	火 (Fire)	300	60
傑尼龜 (Squirtle)	水 (Water)	電 (Electric)	300	60
小火龍 (Charmander)	火 (Fire)	水 (Water)	300	80
皮皮 (Clefairy)	妖精 (Fairy)	鋼 (Steel)	500	60

此外，每隻神奇寶貝皆有專屬的被動技能 (發動攻擊或受到攻擊時觸發)。
下面是被動技能的說明：

- 皮卡丘：受到弱點屬性攻擊時，回復生命值上限的10%(即30點)。先受到傷害再回復生命值，若受到傷害後已死亡，則無法再回復生命值。回復後的血量不可超過初始生命值(300)。
- 妙蛙種子：只要受到弱點屬性攻擊，且該次傷害大於等於當前血量，則強制將血量扣至1點 (即使當前血量為1，受到弱點攻擊後仍維持1點血量)。若受到非弱點屬性的致命攻擊，則正常死亡。此技能無發動次數限制。
- 傑尼龜：發動攻擊後，下次受到的傷害減半。
- 小火龍：受到攻擊後獲得激怒狀態，下次發動攻擊時攻擊力為原來的2倍。此狀態不可疊加。
- 皮皮：發動攻擊時，回復生命值上限的30%(即150點)。回復後的血量不可超過初始生命值(500)。

備註

- 撰寫程式碼時，神奇寶貝名稱和屬性名稱皆使用英文。
- 受到 非弱點屬性 的傷害時，傷害量為攻擊者的攻擊力；受到 弱點屬性 的傷害時，傷害量為攻擊力的 兩倍 。
- 每隻神奇寶貝(包含Boss)的屬性和弱點都只會有一種。
- Boss沒有被動技能，並且只會有一個Boss。
- 所有計算的小數點無條件捨去至整數。



程式碼模板

Pokemon.cpp (實作 TODO 部分)

```

1  #include <vector>
2  #include <string>
3  #include <iostream>
4  #include <cmath>
5  #include <climits>
6  #include <algorithm>
7
8  class Pokemon {
9  protected:
10     std::string name;
11     std::string type;
12     std::string weakness;
13     int HP;
14     int attack;
15 public:
16     Pokemon() = default;
17     virtual ~Pokemon() = default;
18
19     virtual std::pair<int, std::string> go_attack() = 0;
20     virtual void get_attack(int damage, std::string type) = 0;
21     bool dead(){ return HP <= 0; }
22
23     void report(){
24         std::cout << "Name: " << name << std::endl;
25         std::cout << "HP: " << HP << std::endl;
26     }
27 };
28
29 class Boss {
30 private:
31     std::string name;
32     std::string type;
33     std::string weakness;
34     int HP;
35     int attack;
36 public:
37     Boss(std::string name,
38         std::string type,
39         std::string weakness,
40         int HP,
41         int attack)
42         : name(name), type(type), weakness(weakness), HP(HP), attack(attack){}
43     std::pair<int, std::string> go_attack(){
44         // TODO
45     }
46     void get_attack(int damage, std::string type){
47         // TODO
48     }
49     bool dead(){ return HP <= 0; }
50
51     void report(){
52         std::cout << "Name: " << name << std::endl;
53         std::cout << "HP: " << HP << std::endl;
54     }
55 };
56
57 class Pikachu : public Pokemon{
58     // TODO
59 };
60
61 class Bulbasaur : public Pokemon{
62     // TODO
63 };
64
65 class Squirtle : public Pokemon{
66     // TODO
67 };
68
69 class Charmander : public Pokemon{
70     // TODO
71 };
72
73 class Clefairy : public Pokemon{
74     // TODO
75 };

```





main.cpp (不可修改)





```
1  #include "Pokemon.cpp"
2  #include <iostream>
3
4  using namespace std;
5
6  class Manager {
7  private:
8      vector<Pokemon*> pokemons;
9      Boss* boss;
10 public:
11     Manager(){}
12     ~Manager(){
13         delete boss;
14         for(Pokemon* p : pokemons){
15             delete p;
16         }
17         pokemons.clear();
18     }
19     void create_boss(std::string name,
20                     std::string type,
21                     std::string weakness,
22                     int HP,
23                     int attack){
24         boss = new Boss(name, type, weakness, HP, attack);
25     }
26
27     void create_pokemons(){
28         pokemons.push_back(new Pikachu());
29         pokemons.push_back(new Bulbasaur());
30         pokemons.push_back(new Squirtle());
31         pokemons.push_back(new Charmander());
32         pokemons.push_back(new Clefairy());
33     }
34
35     bool pokemon_turn(string pokemon){
36         Pokemon* now;
37         if (pokemon == "Pikachu") {
38             now = pokemons[0];
39         } else if (pokemon == "Bulbasaur") {
40             now = pokemons[1];
41         } else if (pokemon == "Squirtle") {
42             now = pokemons[2];
43         } else if (pokemon == "Charmander") {
44             now = pokemons[3];
45         } else if (pokemon == "Clefairy") {
46             now = pokemons[4];
47         } else {
48             std::cout << "Pokemon not found, please enter again!" << std::endl;
49             return false;
50         }
51         if(now->dead()){
52             cout << "This Pokemon has fainted, please send out another Pokemon!" << e
53             return false;
54         }
55         pair<int, std::string> tmp = now->go_attack();
56         boss->get_attack(tmp.first, tmp.second);
57         return true;
58     }
59
60     bool boss_turn(string pokemon){
61         Pokemon* now;
62         if (pokemon == "Pikachu") {
63             now = pokemons[0];
64         } else if (pokemon == "Bulbasaur") {
65             now = pokemons[1];
66         } else if (pokemon == "Squirtle") {
67             now = pokemons[2];
68         } else if (pokemon == "Charmander") {
69             now = pokemons[3];
70         } else if (pokemon == "Clefairy") {
71             now = pokemons[4];
72         } else {
73             std::cout << "Pokemon not found, please enter again!" << std::endl;
74             return false;
75         }
76         if(now->dead()){
```



```
77         cout << "This Pokemon has fainted, please send out another Pokemon!" << endl;
78         return false;
79     }
80     pair<int, std::string> tmp = boss->go_attack();
81     now->get_attack(tmp.first, tmp.second);
82     return true;
83 }
84
85 void report(){
86     cout << "Boss: " << endl;
87     boss->report();
88     cout << endl;
89     cout << "Pokemon: " << endl;
90     for(int i = 0; i < 5; i++)
91         pokemons[i]->report();
92     cout << endl;
93 }
94
95 bool gameover(){
96     return boss->dead();
97 }
98 };
99
100 int main(){
101     Manager manager;
102     string name;
103     string type;
104     string weakness;
105     int HP;
106     int attack;
107     cin >> name >> type >> weakness >> HP >> attack;
108     manager.create_boss(name, type, weakness, HP, attack);
109     manager.create_pokemons();
110
111     string input;
112     int turn = 0;
113     while(cin >> input){
114         if(input == "0")
115             break;
116         if(input == "Report"){
117             manager.report();
118             continue;
119         }
120         if(turn == 0){
121             if(!manager.pokemon_turn(input))
122                 continue;
123         }
124         else {
125             if(!manager.boss_turn(input))
126                 continue;
127         }
128         if(manager.gameover()){
129             std::cout << "Congratulations! You defeated the BOSS!" << std::endl;
130             break;
131         }
132         turn = 1 - turn;
133     }
134     return 0;
135 }
```

實作內容說明

本作業要實作出 皮卡丘、妙蛙種子、傑尼龜、小火龍 和 皮皮 這五隻神奇寶貝的類別，每個類別包含 Constructor、go_attack 和 get_attack 三個函式。
可依需求新增變數。

Constructor

依照題目說明的表格，設定神奇寶貝的 名稱(name)、屬性(type)、弱點(weakness)、生命值(HP) 以及 攻擊力(attack)。



go_attack()

- 依據 攻擊力 和 被動技能 計算出總攻擊力，並回傳一個 pair，內容包含 { 總攻擊力, 屬性 }。
- 若有觸發其他被動技能，也須一併處理。

ex. 以 小火龍 為例，假設在前一回合有受到boss的攻擊，則此回合攻擊力為

攻擊力 + 攻擊力 * 100% = 80 + 80 * 100% = 160

get_attack(int damage, std::string type)

傳入參數說明

- damage: Boss給予的傷害量
- type: 傷害的屬性

功能說明

- 依據 damage、type 和 被動技能 計算出總傷害量，並從 HP 中減去總傷害量。
- 計算總傷害量時，先判斷是否為弱點屬性的傷害，再套用 被動技能 的公式。
- 總傷害量不會低於 0。
- 若有觸發其他被動技能，也須一併處理。

ex. 以 皮卡丘 為例，假設傳入的 damage 為 80，type 為 Ground，則



1. 受到傷害的屬性為皮卡丘的弱點屬性：總傷害量 = $\text{damage} * 2 = 80 * 2 = 160$
2. 被動技能：回復30點HP

另外需完成 Boss 的 `go_attack` 和 `get_attack` 函式。

`go_attack()`

- 回傳一個 `pair`，內容包含 { 總攻擊力, 屬性 }。

`get_attack(int damage, std::string type)`

傳入參數說明

- `damage`: Boss給予的傷害量
- `type`: 傷害的屬性

功能說明

- 受到傷害時，依據 `damage` 和 `type` 計算出總傷害量，並從 `HP` 中減去總傷害量。

測資輸入說明

首先，新增一個Boss，依序輸入Boss名稱、屬性、弱點、生命值、攻擊力。

```
Clefable Fairy Steel 600 100
```

接著進入戰鬥迴圈。每次輸入一個神奇寶貝名稱，代表依序輪流進行操作。

第1次輸入代表：小智選擇該神奇寶貝對Boss發動攻擊。

第2次輸入代表：Boss選擇該神奇寶貝作為攻擊對象。

第3次輸入代表：小智選擇該神奇寶貝對Boss發動攻擊...

依此類推，直到輸入 "0" 或Boss死亡。

```
Pikachu //小智選擇皮卡丘發動攻擊
Bulbasaur //Boss對妙蛙種子發動攻擊
Clefairy //小智選擇皮皮發動攻擊
Charmander //Boss對小火龍發動攻擊
```

輸入Report，會輸出當前Boss和所有神奇寶貝的血量。

```
Report
```

評分測試

可使用下方指令一次測完所有測資，測試資料有10個

```
HW4 Pokemon.cpp main.cpp
```



如果全部通過，會看到下面畫面：

```
=====
Homework 4: Pokemon Battle
=====
Starting Evaluation...
Testcase 1: ✓ PASSED
Testcase 2: ✓ PASSED
Testcase 3: ✓ PASSED
Testcase 4: ✓ PASSED
Testcase 5: ✓ PASSED
Testcase 6: ✓ PASSED
Testcase 7: ✓ PASSED
Testcase 8: ✓ PASSED
Testcase 9: ✓ PASSED
Testcase 10: ✓ PASSED
=====
Final Score      : 100 / 100
=====
```

在命令後面再加上測試資料的編號，可以單一測某個測試資料：

HW4 Pokemon.cpp main.cpp 3

```
=====
Homework 4: Pokemon Battle
=====
Starting Evaluation... (Target: Testcase 3)
Testcase 3: ✓ PASSED
=====
Final Score      : 10 / 10
=====
```

Final Score 就是你最後的分數，假設 10 個測試資料全通過的話，就是拿滿本次作業分數。

範例測資

Example 1

Input

```
Charizard Fire Water 200 50
Squirtle
Squirtle
Report
Squirtle
```

Output

```
Boss:
Name: Charizard
HP: 80

Pokemon:
Name: Pikachu
HP: 300
Name: Bulbasaur
HP: 300
Name: Squirtle
HP: 275
Name: Charmander
HP: 300
Name: Clefairy
HP: 500

Congratulations! You defeated the BOSS!
```



Example 2

Input

```
Groudon Ground Water 600 140
Pikachu
Pikachu
Bulbasaur
Bulbasaur
Report
0
```

Output

```
Boss:
Name: Groudon
HP: 460

Pokemon:
Name: Pikachu
HP: 50
Name: Bulbasaur
HP: 160
Name: Squirtle
HP: 300
Name: Charmander
HP: 300
Name: Clefairy
HP: 500
```